

Lab 02 Report

Static and Dynamic Arrays, Pointers and Dynamic Memory Allocation

Name: Muhammad Ahmed Raza

Registration No: BSCS-15D-045

Section: BSCS-15D

Course: Data Structures and Algorithms

Lab 02 — Date: 15th September 2026

Task 1 — 1D Array with Pointer Access

Task / Input	Expected	Actual	Pass?
Task 1 Input: 10 20 30 40 50	Total 150, then updated total 152	Total 150, then updated total 152	Pass

Explanation questions

p is a pointer variable that stores the address of sales[0]. It is the address of the first array element.

&p is the address of the pointer variable p itself, not the address it holds. p is a variable in its own right and it lives somewhere in memory too, so &p is a pointer to a pointer (an int**), while p is just an int*.

*p dereferences p and gives the value stored at the address p points to, i.e. sales[0].

*(p + 2) moves the pointer two elements forward (pointer arithmetic scales by sizeof(int)) and then dereferences it, giving sales[2], the third day's value.

Valid indices for sales are 0 to 4, since the array has 5 elements. *(p + 5) would access sales[5], which is one past the last element. That memory does not belong to the array, so reading it is undefined behaviour — it might print garbage, might happen to look fine, or might crash, and none of that is guaranteed.

Task 2 — Dynamic 1D Array

Task / Input	Expected	Actual	Pass?
Task 2 n = 4; marks = 40 50 70 81	Total 241, average 60.25, pass count 3	Total 241, average 60.25, pass count 3	Pass
Task 2 n = 1; mark = 0	Total 0, average 0, pass count 0	Total 0, average 0, pass count 0	Pass
Task 2 n = 0	Error, no allocation or input	Error: n must be between 1 and 10.	Pass

Explanation questions

marks is a pointer variable — it just holds an address and takes up its own small bit of storage (4 or 8 bytes depending on the system). The n integers it points to are a completely separate block on the heap, created by new int[n]. Deleting marks (i.e. delete[] marks) frees that block, but marks itself is a normal local variable that keeps existing until the function ends; it just no longer holds a valid address, which is why we set it to nullptr afterwards.

int marks[n]; is not standard C++ because the size inside [] for a plain array declaration has to be known at compile time — the compiler needs to decide how much stack space to reserve before the program even runs. n is only known once the program is running and the user has typed a value, so the compiler cannot use it. new int[n] works because heap allocation happens at run time, so the size can be a variable.

Task 3 — Fixed 2D Array and Row Pointer

Task / Input	Expected	Actual	Pass?
Task 3 Branch 1: 2 4 6, Branch 2: 1	Branch totals 12, 9; day totals 3,	Branch totals 12, 9; day	Pass

3 5	7, 11	totals 3, 7, 11	
Task 3 Both rows: 0 0 0	Every total is 0	Every total is 0	Pass

Explanation questions

rowPtr + 1 points to the start of the next row, i.e. sales[1]. Because rowPtr's type is int(*)[3], the compiler knows each row is 3 ints wide, so adding 1 advances the address by 3 * sizeof(int) bytes, not just sizeof(int) like a plain int* would. An ordinary int* has no idea rows exist, so it can only step one integer at a time.

int** p = sales; should not be used because sales is a genuine 2D array (a contiguous block of 6 ints), and its name decays to a pointer to its first row — type int(*)[3] — not a pointer to a pointer. An int** is meant to point to a separate int* somewhere, which is not what sales is. Besides not compiling as written, if it were forced to compile it would compute completely wrong addresses when used with pointer arithmetic.

Task 4 — Dynamic 2D Array (Double Pointer)

Task / Input	Expected	Actual	Pass?
Task 4 2x3: 50 60 70 / 80 40 90	Totals 180, 210; top student 2	Totals 180, 210; top student 2	Pass
Task 4 2x2: 40 60 / 50 50	Totals 100, 100; top student 1	Totals 100, 100; top student 1	Pass
Task 4 1x1: 75	Total 75; top student 1	Total 75; top student 1	Pass

Explanation question — partial delete

If only delete[] marks; were executed, that would free the array of row pointers (the block holding marks[0], marks[1], ... marks[rows-1]), but each of the rows themselves — the separate blocks created individually by new int[cols] inside the loop — would still be sitting allocated on the heap with nothing left pointing to them. That memory becomes unreachable and can never be freed for the rest of the program's run, which is a memory leak. Every row has to be deleted first, and only then the row-pointer array.

Memory sketch (2 students, 3 subjects)

marks is a single pointer variable holding the address of a small block of two int* entries. Each of those entries then points to its own separately allocated row of three ints.

```
marks (int**)
|
v
+-----+
| marks[0] (addr A) |----> A: [50][60][70]   <- marks[0][0], marks[0][1],
marks[0][2]
| marks[1] (addr B) |----> B: [80][40][90]   <- marks[1][0], marks[1][1],
marks[1][2]
+-----+
    array of row pointers (2 entries, on the heap)
```

marks is the pointer to the array of row pointers. marks[0] is the pointer to the first row (address A above). marks[0][0] is the actual integer value 50 stored inside that first row.

Task 5 — Expanding a Dynamic Array

Task / Input	Expected	Actual	Pass?
Task 5 n=3; marks 45 60 75; new mark 90	45 60 75 90	45 60 75 90	Pass

Task 5 n=1; mark 0; new mark 100	0 100	0 100	Pass
----------------------------------	-------	-------	------

Explanation question

Changing the variable `n` by itself does not touch the heap at all — it is just an `int` sitting in a register or on the stack. The block that `new int[n]` created has a fixed size that was locked in at the moment it was allocated; the memory manager does not go back and look at `n` again later. Whatever comes right after that block in memory may already belong to something else, so there is no way to safely make it bigger in place. The only way to "grow" it is what the task does: allocate a new, bigger block, copy the old values across, and free the old one.

Task 6 — Finding and Fixing Errors

Errors in the original fragment

1. Loop bound: `for (int i = 0; i <= n; i++)` runs for `i = 0, 1, 2, 3` — four iterations on an array of size 3. The last iteration writes to `values[3]`, which is one element past the end of the allocated block. This is an out-of-bounds write and can silently corrupt whatever memory happens to sit next to the array.
2. `delete values;` frees a block that was created with `new int[n]` using plain `delete` instead of `delete[]`. For arrays this is undefined behaviour — it does not correctly deallocate the whole block the way `delete[]` does, and in general can corrupt the heap's own bookkeeping.
3. `cout << values[0];` runs after the block has already been deleted. Reading through values here is a use of a dangling pointer — the memory may have been handed back to the system or reused for something else, so the value printed is meaningless and the access itself is undefined behaviour.

Corrected program test

Task / Input	Expected	Actual	Pass?
Task 6 Input: 4 5 6	Values: 4 5 6	Values: 4 5 6	Pass

Distinguishing the three problems

An out-of-bounds access reads or writes an index outside the valid range of a still-allocated array — the memory exists, it just is not part of this array. A dangling pointer is a pointer that still holds the address of memory that has already been freed, so the memory it points to no longer belongs to it at all. A memory leak is the opposite kind of mistake — memory that is still allocated but that nothing in the program can reach or free anymore, so it is wasted for the rest of the run.

A program that runs and prints something reasonable does not prove its memory accesses were valid, because undefined behaviour is exactly that — undefined, not guaranteed to fail. An out-of-bounds write might land in unused padding and cause no visible symptom this run, on this compiler, with this input. The same code could crash, print garbage, or behave differently the next time it is compiled, run on another machine, or given slightly different input. Passing a test only shows the output matched for that particular run, not that every memory access was correct.

Academic integrity: I discussed the general approach with classmates but wrote, compiled and tested every program myself.